SAVEETHA SCHOOL OF ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

C PROGRAMMING ASSIGNMENT

Student Submission Template

Student Name	P H Devesh
Register Number	1911260236
Course Code & Name	CSA0209 & C Programming
Slot	B
Assignment / Task Title	C05 – Comprehensive System Design and Implementation Challenge
GitHub Repository URL	https://github.com/DevPH/C-CSA0209
Submission Date	_____

Instructions to Students

1. Complete every section for each assigned task.
2. Write your own explanation, algorithm/pseudocode and program.
3. Run the program and insert a clear screenshot of the output.
4. Include test cases and a short analysis where applicable.
5. If code is maintained on GitHub, provide the correct repository link.

C05 – Comprehensive System Design and Implementation Challenge

Q1. Case Study – District Blood Bank Network (50 Marks)

A district has one central blood bank and several hospital storage units. Each donor record stores donor ID, name, blood group, age, phone number and last-donation date. Each stock record stores a blood group and available units. The system must register donors, find eligible donors of a requested group, update stock after donations and issues, and preserve data in files so it remains available after shutdown.

PART A – DESIGN & PSEUDOCODE (25 Marks)

All pseudocode follows the SIMATS Engineering Standard for Algorithmic Pseudocode.

TASK A1 – DECOMPOSE THE PROGRAM INTO FUNCTIONS (7 Marks)

A1.1 – Identify the Functions | 2 Marks

The program is divided into independent functions. Each function has one clear responsibility and communicates through parameters and return values.

```
main()
 ■■■ load_data()
 ■    ■■■ load_donors()
 ■    ■■■ load_stock()
 ■■■ register_donor()
 ■    ■■■ validate_donor()
 ■    ■■■ add_donor()
 ■    ■■■ save_donors()
 ■■■ find_eligible_donors()
 ■    ■■■ is_eligible()
 ■    ■■■ display_donor()
 ■■■ record_donation()
 ■    ■■■ validate_group()
 ■    ■■■ update_stock()
 ■    ■■■ save_stock()
 ■■■ issue_blood()
 ■    ■■■ check_available_units()
 ■    ■■■ update_stock()
 ■    ■■■ save_stock()
 ■■■ cleanup()
      ■■■ free_donor_list()
```

A1.2 – Design the Interfaces | 3 Marks

```
int load_donors(const char *filename, DonorList *list);
int save_donors(const char *filename, const DonorList *list);
int load_stock(const char *filename, Stock stock[8]);
int save_stock(const char *filename, const Stock stock[8]);

int validate_donor(const Donor *donor);
int is_eligible(const Donor *donor, Date today);
int add_donor(DonorList *list, const Donor *donor);
int find_eligible_donors(const DonorList *list, const char *group,
                         Date today, int *youngest_donor_id);
int update_stock(Stock stock[8], int group_index, int delta);
Stock *find_lowest_stock(Stock stock[8], int count);
void free_donor_list(DonorList *list);
```

Scalar values such as group index and stock delta are passed by value because the function needs only a copy. Donor records and strings are passed by const pointer when they are read-only, avoiding unnecessary structure copies. DonorList and stock arrays are passed by pointer when the function must update the caller's data. The const qualifier documents that a function must not modify an input object.

Q1. Case Study – District Blood Bank Network (continued)

A1.3 – Return More Than One Value | 2 Marks

Alternative 1 – output pointer: Return the number of eligible donors as the function result and store the youngest eligible donor ID through an output parameter.

```
int find_eligible_donors(const DonorList *list, const char *group,
                         Date today, int *youngest_donor_id);

/* Caller */
int youngest_id = -1;
int count = find_eligible_donors(&donors, "O+", today, &youngest_id);
```

Alternative 2 – return a structure:

```
typedef struct {
    int eligible_count;
    int youngest_donor_id;
} EligibilityResult;

EligibilityResult find_eligible(const DonorList *list,
                                const char *group, Date today);
```

The pointer-output design is compact and keeps the integer return value available for the count or an error status. The structure-return design groups related results into one meaningful object and is easier to extend with more statistics. Both use O(1) additional memory and do not change the O(n) donor scan.

Alternative using global variables: The donor list and count could be global and accessed by every function. The running time and memory complexity would remain unchanged because the same records are searched and stored. However, functions would depend on hidden shared state, testing would require resetting globals, and reuse with multiple blood-bank datasets would become difficult. Passing parameters makes dependencies explicit and improves modularity.

Evidence expected: structure chart, justified prototypes and two designs for returning multiple values.

TASK A2 – DESIGN WITH RECURSION (5 Marks)

A2.1 – Recursive Design | 2 Marks

The digital root is obtained by repeatedly adding the digits until one digit remains. Exponentiation is implemented using both linear recursion and divide-and-conquer recursion.

```
DIGITAL_ROOT(n)
    IF n < 10
        RETURN n
    ENDIF
    sum ← 0
    WHILE n > 0
        sum ← sum + (n MOD 10)
        n ← n DIV 10
    ENDWHILE
    RETURN DIGITAL_ROOT(sum)

POWER_LINEAR(base, exp)
    IF exp = 0
        RETURN 1
    ENDIF
    RETURN base × POWER_LINEAR(base, exp - 1)

POWER_FAST(base, exp)
    IF exp = 0
        RETURN 1
    ENDIF
    half ← POWER_FAST(base, exp DIV 2)
    IF exp MOD 2 = 0
        RETURN half × half
    ELSE
        RETURN base × half × half
    ENDIF
```

For digital root, the base case is a one-digit value (n less than 10); the recursive case sums the digits and calls DIGITAL_ROOT again. For both power functions, exp=0 is the base case. Linear recursion decreases exp by one; fast recursion halves exp.

Q1. Case Study – District Blood Bank Network (continued)

A2.2 – Trace the Calls | 2 Marks

For input 9875, the digital-root trace is 9875 → 9+8+7+5=29 → 2+9=11 → 1+1=2. The recursive calls are DIGITAL_ROOT(9875), DIGITAL_ROOT(29), DIGITAL_ROOT(11), DIGITAL_ROOT(2). The deepest call returns 2 and each earlier call returns that result.

For POWER_FAST(2,5), the call chain is POWER_FAST(2,5) → POWER_FAST(2,2) → POWER_FAST(2,1) → POWER_FAST(2,0). The base call returns 1. Returning upward gives 2, then 4, then 32. The maximum active depth is proportional to log■(n).

Call-count recurrence: linear power has T(n)=T(n–1)+1 with T(0)=1, so it makes n+1 calls and uses O(n) time and O(n) stack. Halving power has T(n)=T(■n/2■)+1 with T(0)=1, so it makes Θ(log n) calls and uses O(log n) stack.

A2.3 – Recursion Risks | 1 Mark

If the base case is missing or never reached, recursive calls continue until the call stack is exhausted, causing stack overflow. A loop should replace recursion when the input can be very large, when constant stack space is important, or when recursion does not improve clarity.

Alternative digital-root analysis: A loop repeatedly sums digits in O(d) time and O(1) space, where d is the number of digits. For positive n, the direct formula 1 + (n–1) % 9 gives O(1) time and O(1) space; n=0 must be handled separately. Recursive digit reduction uses additional call-stack space.

Evidence expected: correct recursive designs, base cases, call traces, recurrences and an explanation of recursion risks.

TASK A3 – DESIGN WITH POINTERS (5 Marks)

A3.1 – Pointer Operations | 2 Marks

```
int stock[8] = {12, 30, 18, 25, 9, 14, 6, 20};
int *p = stock;

/* p + 3  : address of stock[3] */
/* *(p+3) : value at stock[3], which is 25 */
/* p[3]   : equivalent to *(p+3), also 25 */
/* *p++   : reads stock[0] (12), then advances p to stock[1] */
```

Illustrative memory diagram (assuming 4-byte int):

```
Index:       0       1       2       3       4       5       6       7
Value:      12      30      18      25       9      14       6      20
Address: 1000    1004    1008    1012    1016    1020    1024    1028

Initially p = 1000 (address of stock[0]).
p + 3 = 1012 (address of stock[3]).
```

Q1. Case Study – District Blood Bank Network (continued)

A3.2 – Pointers and Functions | 2 Marks

The function returns a pointer to the lowest-stock element in the caller-owned array. The caller can use the returned pointer to update that element directly.

```
int *find_lowest_stock(int stock[], int n)
{
    if (n <= 0) return NULL;

    int *lowest = &stock[0];
    for (int i = 1; i < n; ++i) {
        if (stock[i] < *lowest)
            lowest = &stock[i];
    }
    return lowest;
}

/* Example caller */
int *p_low = find_lowest_stock(stock, 8);
if (p_low != NULL)
    (*p_low)++;
```

Returning the address of a local variable is wrong because the local object stops existing when the function returns. The returned pointer would be dangling. The array passed by the caller remains alive, so a pointer to one of its elements is valid while the caller-owned array remains valid.

A3.3 – Command-Line Interface | 1 Mark

```
bloodbank find O+
bloodbank issue A- 2
bloodbank donate 7001 O+ 1
```

The program checks argc before reading argv. It validates the command, blood group, donor ID and positive numeric units. Missing, extra or invalid arguments produce a usage message on stderr and a non-zero exit status.

Alternative index design: The function may return the index of the lowest-stock element instead of a pointer. The caller then writes stock[index]. Both methods scan n elements in O(n) time and use O(1) extra space. An index can be applied to the array after realloc moves it; a pointer obtained before reallocation may become invalid. The index approach is therefore safer when the array can be relocated.

Evidence expected: correct pointer expressions with memory diagram, safe pointer-returning function and command-line design.

TASK A4 – DESIGN DYNAMIC MEMORY USE (4 Marks)

A4.1 – Growable Donor List | 2 Marks

The number of donors is unknown in advance. A dynamic array starts with a small capacity and doubles when full.

```
ADD_DONOR(list, donor)
    IF list.size = list.capacity
        new_capacity ← 2 IF capacity = 0 ELSE 2 × capacity
        temp ← REALLOC(list.data, new_capacity × SIZEOF(Donor))
        IF temp = NULL
            RETURN FAILURE
        ENDIF
        list.data ← temp
        list.capacity ← new_capacity
    ENDIF
    list.data[list.size] ← donor
    list.size ← list.size + 1
    RETURN SUCCESS
```

The old allocation remains valid if realloc fails, so the returned pointer must first be stored in a temporary variable. Only after success should list.data and capacity be updated.

Q1. Case Study – District Blood Bank Network (continued)

A4.2 – Memory Errors | 2 Marks

A memory leak occurs when allocated donor storage is no longer reachable and cannot be freed. A dangling pointer occurs when a pointer is used after free or after realloc has moved the allocation. A double free occurs when the same allocation is released twice. The design prevents these errors by retaining the original pointer on realloc failure, maintaining one owner, freeing the list once in cleanup, and setting the pointer to NULL after release.

Growth strategy comparison: Doubling gives O(n) total copying across n insertions and amortized O(1) append. Increasing capacity by a fixed 10 may copy 10+20+…+n elements, resulting in $O(n^2)$ total copying. Fixed increments leave at most 9 spare slots, whereas doubling may leave nearly half the allocated capacity unused immediately after growth.

Evidence expected: growable-array design with failure handling and correct explanations of memory leak, dangling pointer and double free.

TASK A5 – DESIGN THE FILE STORAGE (4 Marks)

A5.1 – Text or Binary? | 2 Marks

Donor records are stored as a text file with one donor per line, written using fprintf. Text is readable and can be inspected or edited easily. The stock file is stored as binary fixed-size records using fwrite. Binary storage is compact and supports direct positioning by record index. Binary files may be less portable because integer sizes, byte order and structure padding can differ across systems.

A5.2 – Update in Place | 2 Marks

```
UPDATE_UNITS(fp, index, units)
    IF index < 0: RETURN FAILURE
    IF FSEEK(fp, index × SIZEOF(Stock), SEEK_SET) fails:
        RETURN FAILURE
    IF FREAD(&record, SIZEOF(Stock), 1, fp) != 1:
        RETURN FAILURE
    record.units ← units
    IF FSEEK(fp, -SIZEOF(Stock), SEEK_CUR) fails:
        RETURN FAILURE
    IF FWRITE(&record, SIZEOF(Stock), 1, fp) != 1:
        RETURN FAILURE
    RETURN SUCCESS
```

A fixed-size binary record can be updated in O(1) time relative to the number of records. Text lines may have different lengths. Changing one numeric value can alter the line length, so later data may need to be shifted or the whole file rewritten. Rewriting a text file for one update takes O(n) time.

Evidence expected: justified file designs and correct pseudocode for an in-place update.

Q1. Case Study – District Blood Bank Network (continued)

PART B – IMPLEMENTATION & EXPERIMENTAL INVESTIGATION (25 Marks)

Part B follows the process: Implement → Instrument → Stress → Measure → Compare → Interpret. Execution screenshots alone are insufficient; include test cases and measured evidence.

TASK B1 – IMPLEMENT AND INSTRUMENT THE RECURSIVE FUNCTIONS (7 Marks)

B1.1 – Validate the Implementation | 2 Marks

The following C program implements digital root, linear recursive power and halving recursive power. The global counter records the number of recursive calls for the power functions.

```
#include <stdio.h>
long long calls = 0;

long long power_linear(long long base, int exp)
{
    ++calls;
    if (exp == 0) return 1;
    return base * power_linear(base, exp - 1);
}

long long power_fast(long long base, int exp)
{
    ++calls;
    if (exp == 0) return 1;
    long long half = power_fast(base, exp / 2); /* call once */
    if (exp % 2 == 0) return half * half;
    return base * half * half;
}

int digital_root(int n)
{
    if (n < 10) return n;
    int sum = 0;
    while (n > 0) {
        sum += n % 10;
        n /= 10;
    }
    return digital_root(sum);
}

int main(void)
{
    int tests[] = {0, 1, 9875, 123456};
    for (int i = 0; i < 4; ++i)
        printf("digital_root(%d) = %d\n",
               tests[i], digital_root(tests[i]));

    calls = 0;
    long long a = power_linear(2, 30);
    printf("linear: %lld in %lld calls\n", a, calls);

    calls = 0;
    long long b = power_fast(2, 30);
    printf("fast: %lld in %lld calls\n", b, calls);
    return 0;
}
```

The tests 0 and 1 verify the base case. The input 9875 produces 2 after repeated digit summation. The input 123456 produces 3. Both power algorithms return 1073741824 for 2³■. Linear recursion makes 31 calls; halving recursion makes 6 calls.

B1.2 – Compare with Your Design | 1 Mark

The C code follows the pseudocode. The recursive result for half the exponent is stored once in half and reused. This avoids making the same recursive call twice and preserves logarithmic call growth.

Q1. Case Study – District Blood Bank Network (continued)

B1.3 – Count the Calls | 2 Marks

For linear recursion, calls = n+1. For halving recursion, calls = floor(log■ n)+1 for positive n with the shown base case.

Exponent n	Linear calls	Halving calls
10	11	5
100	101	7
1,000	1,001	10
10,000	10,001	14

If half × half is replaced with two calls to power_fast(base, exp/2), the same subproblem is evaluated twice. The recurrence becomes T(n)=2T(n/2)+O(1), giving O(n) calls instead of O(log n).

B1.4 – Find the Limit | 2 Marks

Increase n for the linear recursive version in a controlled test until it fails, and record the largest successful value and first failing value. The failure is typically stack overflow because each recursive call needs a stack frame. The exact limit depends on compiler, operating system, stack limit and build settings; it must be measured on the student's computer. Repeat with an iterative version. The iterative square-and-multiply method takes O(log n) time and O(1) stack space, while recursive halving uses O(log n) stack space.

TASK B2 – ENGINEER ADVERSARIAL INPUTS (5 Marks)

An unbounded scanf("%s") can overflow a fixed-size name array when a longer string is supplied. For char name[20], scanf("%19s", name) limits input to 19 characters plus the null terminator. fgets(name, sizeof name, stdin) is safer because it limits the number of bytes read; the program should detect and reject an overlong line.

```
char name[20];
if (fgets(name, sizeof name, stdin) == NULL) {
    perror("name input");
    return 1;
}
if (strchr(name, '\n') == NULL) {
    int ch;
    while ((ch = getchar()) != '\n' && ch != EOF) { }
    fprintf(stderr, "Name too long\n");
    return 1;
}
name[strcspn(name, "\n")] = '\0';
```

For a missing or read-only file, check fopen before using the file pointer; call perror() and return a controlled error instead of crashing. Test command-line cases with no arguments, too many arguments and an invalid blood group. Compile with gcc -std=c11 -Wall -Wextra -fsanitize=address -g program.c -o program, then run normal and adversarial tests. Report any sanitizer or Valgrind errors found and fixed.

Alternative: getline reads a line of length L in O(L) time and allocates O(L) heap space. The caller owns the allocated buffer and must free it.

Q1. Case Study – District Blood Bank Network (continued)

TASK B3 – TEXT FILE VS BINARY FILE (8 Marks)

B3.1 – Correctness on Small Data | 2 Marks

Write and read back 10 donor records in both a text file and a binary file. Compare every field of each record after reading: donor ID, name, blood group, age, phone number and last-donation date. Matching all fields verifies that serialization and deserialization preserve the records.

B3.2 – Increase the Data Size | 2 Marks

Repeat the experiment for 1,000, 10,000 and 100,000 donors. Measure file size and elapsed write/read time for each format. Fill the table using measurements from the actual computer used for submission.

Donors	Text size (bytes)	Binary size (bytes)	Text write/read (ms)	Binary write/read (ms)
1,000	_____	_____	_____	_____
10,000	_____	_____	_____	_____
100,000	_____	_____	_____	_____

B3.3 – Update One Record | 2 Marks

Measure the time to update the 50,000th record in place in the binary file using fseek, fread and fwrite. For the text file, read all records and write a replacement file with the target record changed, then replace the original. Record both timings using the same test data and machine.

B3.4 – Engineering Trade-Off | 2 Marks

Text files are human-readable and portable, but require parsing and are difficult to update in place when line lengths vary. Binary files are compact and support direct fixed-record updates, but their layout may not be portable across machines due to byte order, integer representation and structure padding. Text is suitable for donor master records that need inspection; binary is suitable for fixed-size stock records requiring direct updates. A production multi-hospital system should consider a database with transactions and concurrency control.

Completed stock update function (TODO 1)

```
#include <stdio.h>
typedef struct { int group_code; int units; } Stock;

int update_units(FILE *fp, int index, int units)
{
    Stock s;
    if (fp == NULL || index < 0 || units < 0) return 0;
    if (fseek(fp, (long)index * (long)sizeof(Stock), SEEK_SET) != 0)
        return 0;
    if (fread(&s, sizeof s, 1, fp) != 1) return 0;
    s.units = units;
    if (fseek(fp, -(long)sizeof(Stock), SEEK_CUR) != 0) return 0;
    if (fwrite(&s, sizeof s, 1, fp) != 1) return 0;
    return fflush(fp) == 0;
}
```

The update of record 9 fails because the sample file contains only five records (valid indices 0–4). The seek positions beyond the available records and fread returns 0. The function returns before writing, so no existing record is modified. Writing the whole array with one fwrite reduces library calls from n to one, but still transfers O(n) records and requires O(n) memory to hold the array; one-record-at-a-time writing uses O(1) extra memory.

Q1. Case Study – District Blood Bank Network (continued)

TASK B4 – FINAL ENGINEERING DECISION (5 Marks)

The district connects 25 hospitals, has 100 times the present donor count and needs a nightly report of donors eligible to donate in the next week.

B4.1 – Implement a Practical Strategy | 2 Marks

Load donor records from the file into a growable donor list. Scan the list once, evaluate the configured eligibility rule for the next seven days, and write matching donors to the report. The scan takes O(n) time and the growable list uses O(n) memory. Measure the actual allocation using capacity × sizeof(Donor), along with the time taken to load and generate the report.

B4.2 – Check Memory Safety | 1 Mark

Free all allocated memory before program exit. A single cleanup function should release the donor array and reset the pointer, size and capacity. All error paths after allocation must also reach cleanup. Include Valgrind summary or AddressSanitizer output showing that no invalid access, use-after-free, double-free or memory leak remains.

B4.3 – Production Decision | 2 Marks

For a dataset that fits comfortably in RAM, loading all donor records gives simple repeated scans and avoids repeated disk access. If the 100× growth exceeds available memory, read the file in batches for the nightly report. If repeated ID lookups are common, maintain a sorted index file containing donor ID and file offset; binary search takes O(log n) index comparisons and uses O(1) extra working memory, with corresponding file seeks. The decision should be based on measured memory and runtime, not assumption alone.

Completed growable-list function (TODO 1)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct { int id; char group[4]; } Donor;
typedef struct { Donor *data; int size, capacity; } DonorList;

int add_donor(DonorList *list, int id, const char *group)
{
    if (list == NULL || group == NULL) return 0;

    if (list->size == list->capacity) {
        int new_capacity = (list->capacity == 0)
                         ? 2 : list->capacity * 2;
        Donor *p = realloc(list->data,
                           (size_t)new_capacity * sizeof *list->data);
        if (p == NULL) return 0;
        list->data = p;
        list->capacity = new_capacity;
    }

    list->data[list->size].id = id;
    strncpy(list->data[list->size].group, group, 3);
    list->data[list->size].group[3] = '\0';
    list->size++;
    return 1;
}

int main(void)
{
    DonorList list = {NULL, 0, 0};
    const char *groups[] = {"O+", "A+", "B-", "AB+", "O-"};

    for (int i = 0; i < 5; ++i) {
        if (!add_donor(&list, 7001+i, groups[i])) {
            perror("realloc");
            free(list.data);
            return 1;
        }
        printf("added %d (%s): size = %d, capacity = %d\n",
               list.data[i].id, list.data[i].group,
               list.size, list.capacity);
    }
    free(list.data);
    return 0;
}
```

Expected output

```
added 7001 (O+): size = 1, capacity = 2
added 7002 (A+): size = 2, capacity = 2
added 7003 (B-): size = 3, capacity = 4
added 7004 (AB+): size = 4, capacity = 4
added 7005 (O-): size = 5, capacity = 8
```

The realloc result is stored in temporary pointer p because realloc returns NULL on failure while the original allocation remains valid. Assigning directly to list->data could lose the only reference to the old allocation. A sorted external index may reduce RAM to O(1) working space, but each lookup needs O(log n) index searches and disk reads.

Q1. Case Study – District Blood Bank Network (continued)

Alternative concept and complexity analysis

For donor lookup, loading all records into memory allows an O(n) scan for a matching donor group, with O(n) memory for the donor list. A separate sorted index of donor IDs and file offsets uses O(n) storage on disk and O(1) additional RAM while searching; binary search needs O(log n) comparisons and file seeks. For a nightly full report, a sequential file scan is O(n) and can be performed in batches to limit RAM.

For dynamic arrays, doubling capacity gives amortized O(1) append and O(n) total copying. Fixed-size increments can cause $O(n^2)$ total copying. For stock records, fixed-size binary records permit O(1) in-place update by offset, while rewriting a text file takes O(n). For recursive exponentiation, linear recursion is O(n) time/O(n) stack, while halving recursion is O(log n) time/O(log n) stack; iterative square-and-multiply keeps O(1) stack.

Conclusion

The District Blood Bank Network design separates donor registration, eligibility checks, stock management and file persistence into clear functions. Pointer parameters allow safe updates to caller-owned data, recursion demonstrates base-case and call-stack behaviour, a doubling dynamic array supports an unknown donor count, and text and binary files are selected according to readability and update requirements. The final strategy must be supported by measured performance and memory-safety evidence. A production deployment should add transaction-safe shared storage, access control, audit logs, backups and medically approved donor-eligibility rules.

Experimental evidence to attach

1. Terminal screenshots for digital-root and power-function test cases.
2. Recursive call-count results for n = 10, 100, 1,000 and 10,000.
3. Largest successful and first failing recursion input, plus iterative comparison.
4. Adversarial long-name, missing/read-only file and invalid-command tests.
5. Text/binary file sizes, write/read timings and record-50,000 update timings.
6. AddressSanitizer or Valgrind output showing memory-safety verification.
7. Growable-list output and memory/time measurements for the nightly report.

Note: The assignment asks for machine-dependent experimental measurements. The measurement table is intentionally left blank until the programs are run on the student's own system; actual values and screenshots should be inserted rather than fabricated.